API Security Risk Analysis Report

Target Information

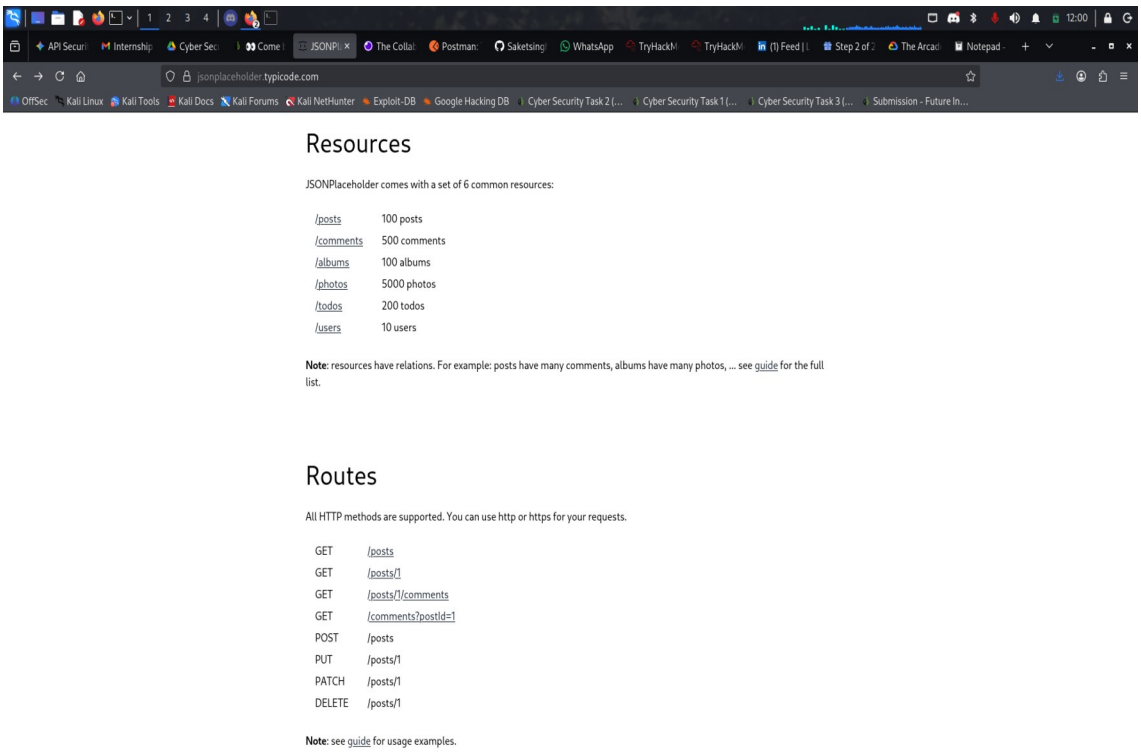
- **Target API:** https://jsonplaceholder.typicode.com
- **Tools Used:** Postman(GET, POST), Browser DevTools(For Documentation)
- **Methodology:** OWASP API Security Top 10 (2023)
- **Author:** Saket Singh

INDEX:

Section	Content
1. Project Overview	Target API, Tools Used, and Methodology
2. Reconnaissance Analysis	Publicly Accessible Detailed API Blueprint
3. Technical Vulnerabilities	**Finding 1: Mass Assignment (High Severity)**
	Finding 2: Excessive Data Exposure (Medium Severity)
	Finding 3: Unrestricted Resource Consumption (Medium Severity)
	Finding 4: Security Misconfiguration (Low Severity)
4. Positive Observations	Proper Handling of Non-Existent Resources (BOLA Prevention)
	Consistent Data Format Enforcement (JSON)
5. Conclusion	Final Summary of Security Posture

Finding 1. Website Documentation Analysis

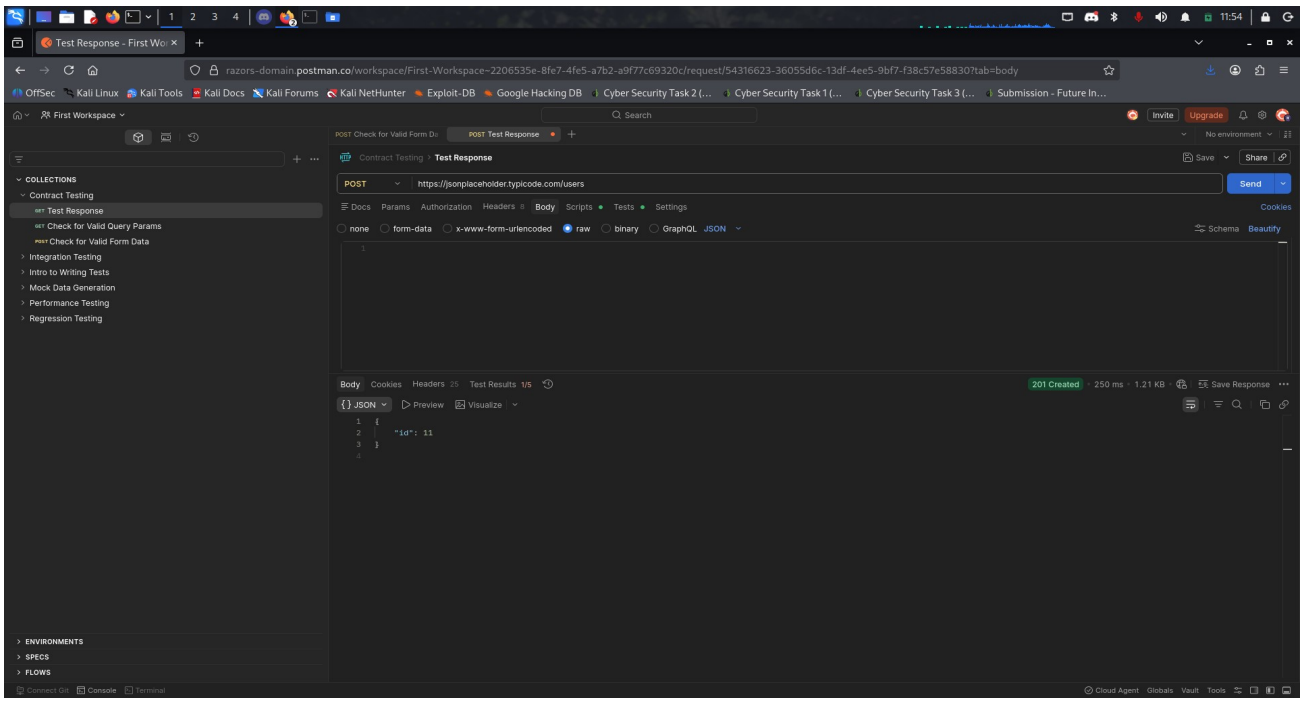
- **Finding: Publicly Accessible Detailed API Blueprint**
    - **Vulnerability Type:** Information Disclosure / Security Misconfiguration.
    - **Severity: Low**.
    - **Evidence:**

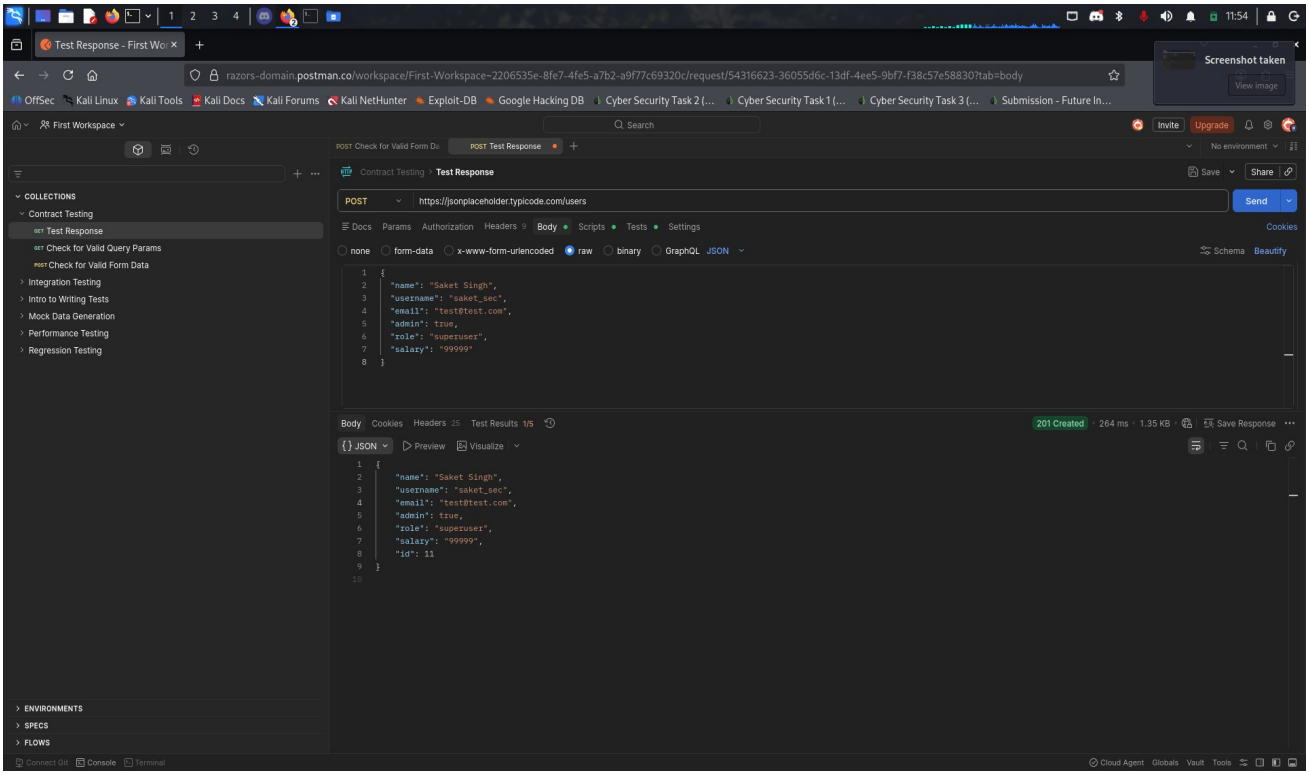


- **Description:** The API provides a full public map of all available routes (`GET`, `POST`, `PUT`, `DELETE`) and resources (`/posts`, `/comments`, `/users`).
- **The Risk:** While necessary for a demo/public API, in a private SaaS environment, this is "leaked documentation." It gives an attacker a complete "Attack Surface Map" without them having to do any manual discovery.
- **Business Impact:** It significantly speeds up the "Reconnaissance" phase for an attacker, allowing them to target sensitive endpoints (like `PUT /users/1`) immediately.
- **Remediation**: In production environments, API documentation (like Swagger/OpenAPI) should be behind authentication or restricted to internal developer networks (VPN).

Finding 2: Mass Assignment (Bypassing Internal Logic)

- **Vulnerability Type:** OWASP API6:2023 - Mass Assignment
- **Severity: High**
- **Evidence:**



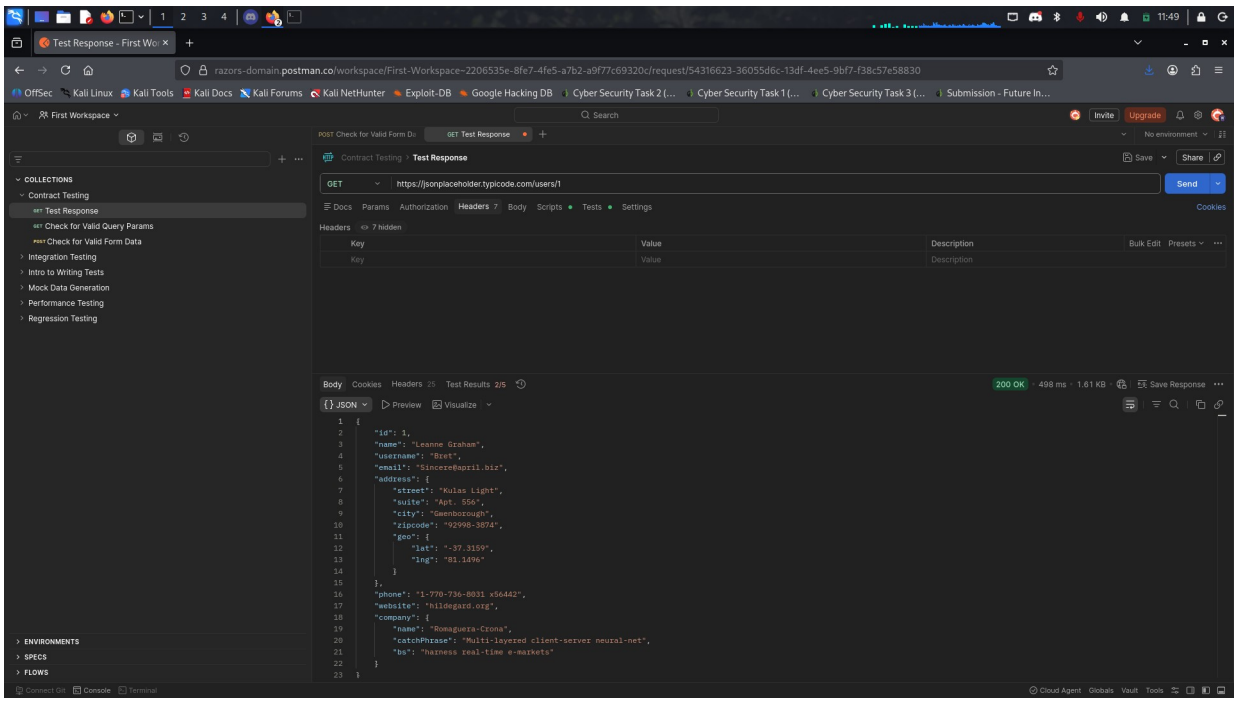


- **Description:** The API endpoint for creating a user blindly accepts all parameters provided in the JSON body. During testing, I successfully injected administrative fields (`admin`, `role`, `salary`) that should typically be restricted to internal system logic or authorized administrators.
- **Business Impact:** An attacker could perform **Privilege Escalation**, upgrading their own account permissions or modifying sensitive data like financial balances or system roles without authorization.

- **Remediation:** Implement a **Whitelist/Allow-list** approach using Data Transfer Objects (DTOs). The backend should only bind specific, non-sensitive fields (e.g., `name`, `username`) and ignore any unauthorized parameters.

Finding 3: Excessive Data Exposure (Information Disclosure)

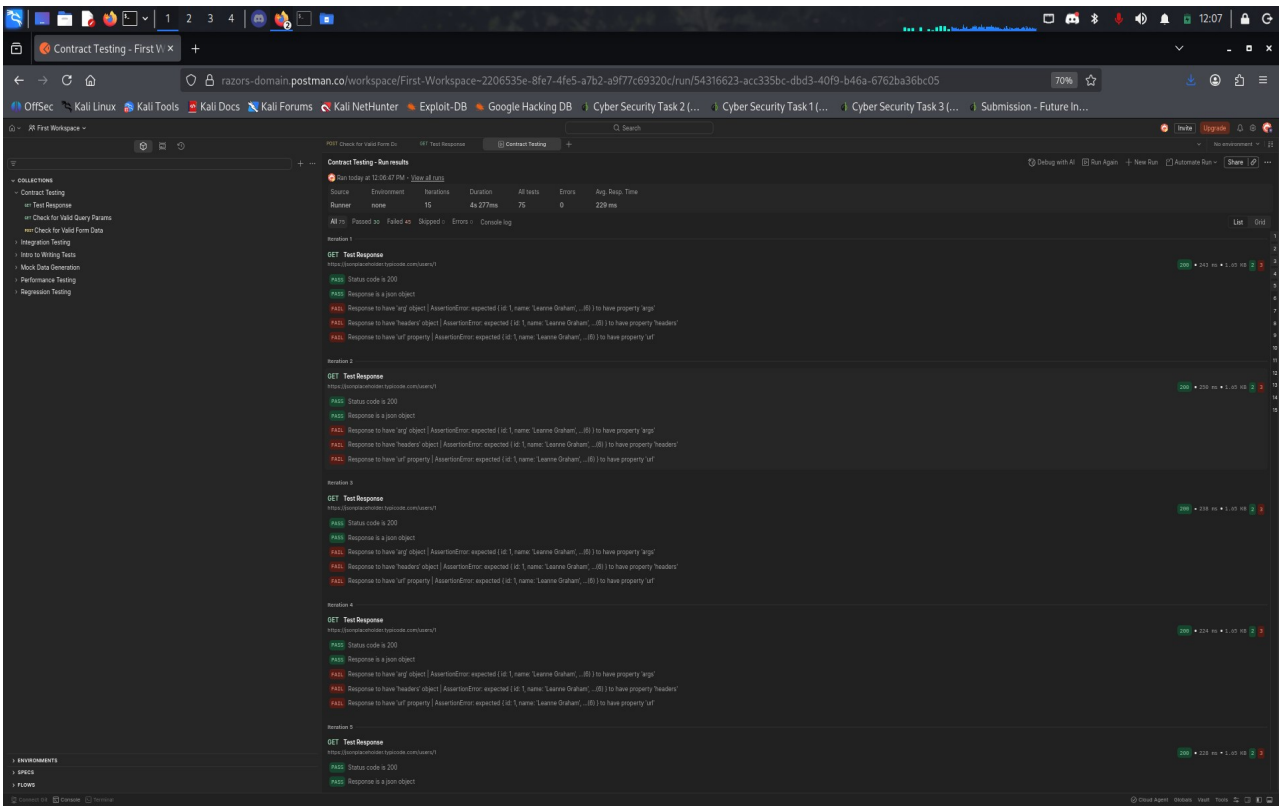
- **Vulnerability Type:** OWASP API3:2023 - Broken Object Property Level Authorization
- **Severity: Medium**
- **Evidence:**



- **Description:** The API response returns the full data object stored in the database, including sensitive metadata and precise geolocation coordinates (`lat`, `lng`). This data is sent to the client even though it may not be necessary for the user interface.
- **Business Impact:** This increases the risk of **Stalking or Physical Security threats** if user locations are leaked. It also provides attackers with detailed intelligence about the internal data structure and metadata, which can be used to plan more complex attacks.
- **Remediation:** Implement **Response Filtering**. The API should only return the specific fields required by the frontend application. Use an abstraction layer to strip out sensitive coordinates and internal metadata before the response is sent.

Finding 4: Lack of Resources & Rate Limiting (DoS Risk)

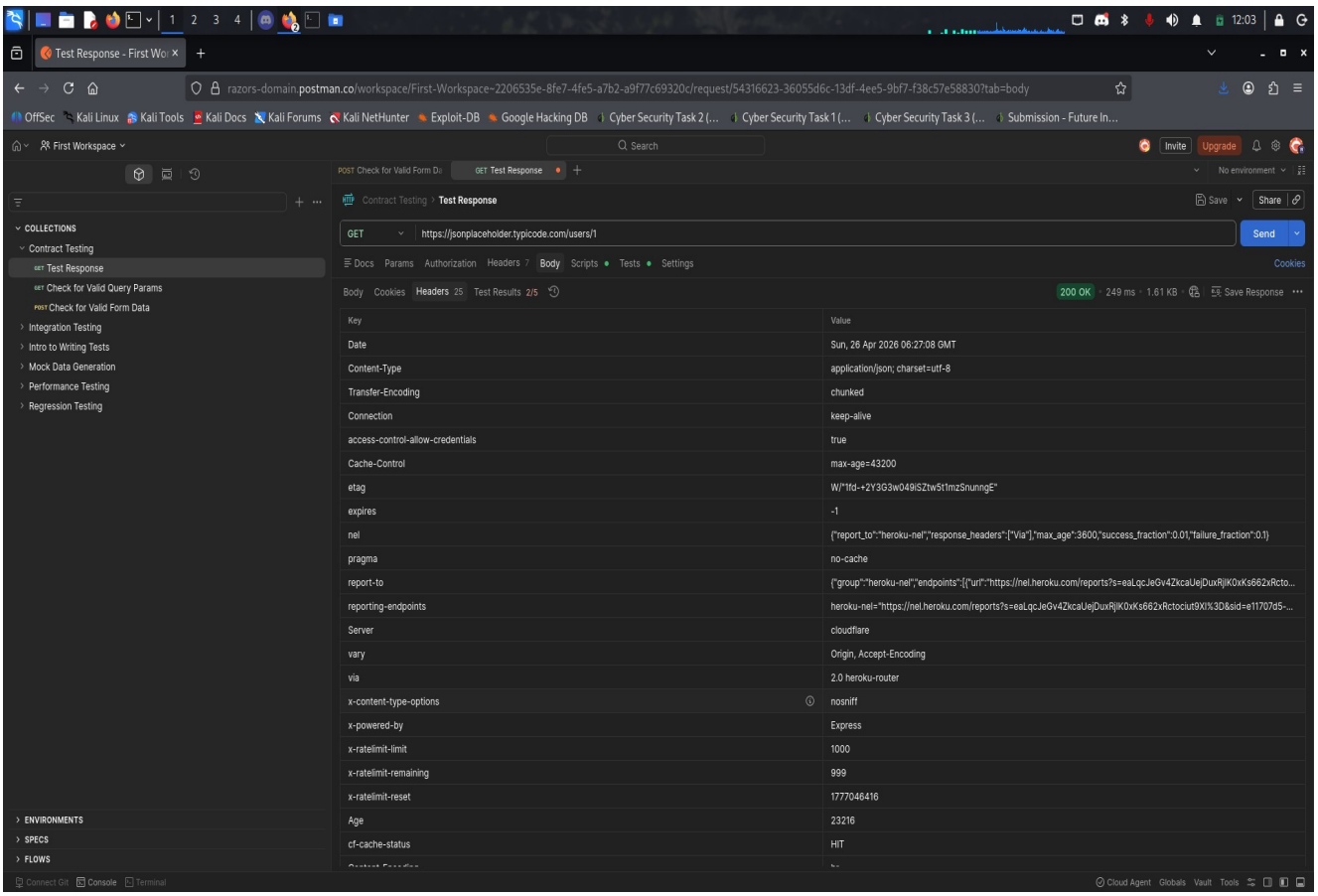
- **Vulnerability Type:** OWASP API4:2023 - Unrestricted Resource Consumption
- **Severity: Medium**
- **Evidence:**



- **Description:** The API does not enforce rate limiting on public endpoints. During testing, I was able to send multiple rapid requests without encountering a `429 Too Many Requests` status code or any form of throttling.
- **Business Impact:** The API is vulnerable to **Resource Exhaustion**. An attacker could automate a script to scrape the entire database or flood the server with requests, leading to a Denial of Service (DoS) for legitimate users.
- **Remediation:** Implement **Rate Limiting** at the API Gateway or Middleware level. Establish a threshold (e.g., 100 requests per minute per IP address) and return a `429` error when the limit is exceeded.

Finding 5: Missing Security Headers (Defense-in-Depth)

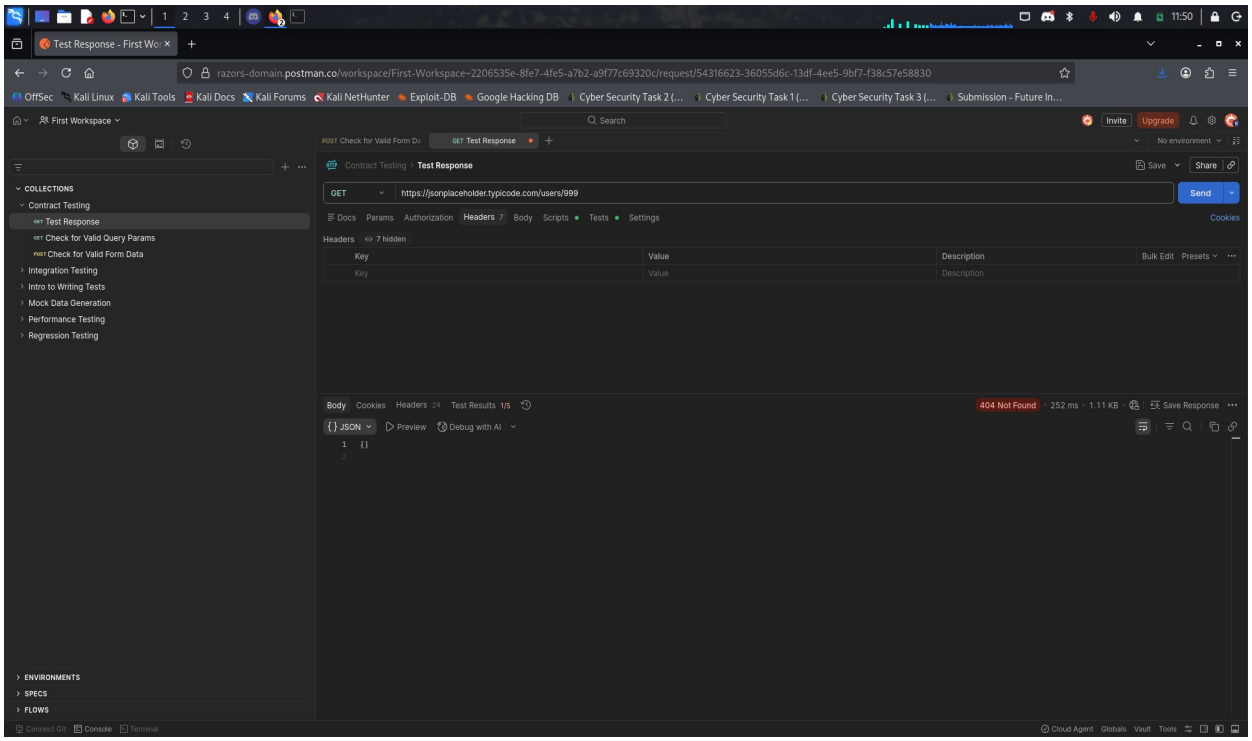
- **Vulnerability Type:** Security Misconfiguration
- **Severity: Low**
- **Evidence:**



- **Description:** Inspection of the HTTP response headers revealed that several industry-standard security headers are missing or improperly configured (e.g., `Content-Security-Policy`, `X-Frame-Options`).
- **Business Impact:** This reduces the "Defense-in-Depth" posture. While not a direct exploit, the lack of these headers makes it easier for attackers to perform Cross-Site Scripting (XSS) or Clickjacking attacks on users of the API.
- **Remediation:** Configure the web server to include security-hardened headers such as `Strict-Transport-Security` (HSTS), `X-Content-Type-Options: nosniff`, and a restrictive `Content-Security-Policy` (CSP).

Finding 6. Positive Security Findings (The "Good" News)

- **Finding: Proper Handling of Non-Existent Resources (BOLA Prevention)**
    - **Observation:** When requesting a non-existent user (e.g., `/users/999`), the API correctly returns a `404 Not Found` with an empty object `{}`.



- **Why it's good:** It doesn't leak whether the user *used* to exist or provide "hints" about the database structure. This is a solid implementation of **Broken Object Level Authorization** defense for invalid IDs.

- **Finding: Consistent Data Format**
  - **Observation:** The API strictly enforces JSON for both requests and responses.
  - **Why it's good:** Consistency makes it easier to implement automated security filters and reduces the risk of "Content-Type" confusion attacks where a hacker tries to send XML to a JSON endpoint to bypass filters.